MODULE 30 · DOMAIN 4 - EVAL, TESTING AND DEBUGGING

Debugging and Error Handling

Error types and what each one means, HTTP 200 that is not success, traces worth keeping, isolating the integration layer from the model, and recovery strategies by design.

DOMAIN 4 of 8 EXAM WEIGHT 2.6% MODULE 30 of 36

admin@sthithapragna.com

A good mechanic does not replace the engine because of a rattle. They listen, find the loose part, and tighten that.

Four families of error

FAMILY	LOOKS LIKE	FIRST MOVE
HTTP errors	4xx, 5xx, dropped connections	Retry the transient; fix your request
Stop reasons	HTTP 200, but truncated or refused	Branch on stop_reason before parsing
Integration faults	Bad parse, missing context, tool errors	Inspect the request and raw output
Model-output faults	Wrong label, unsupported claim	Cluster cases; prompt, examples, evals

ON THE EXAM

Name the family before you touch anything. Each one has a different fix.

HTTP errors: retry, fix or stop

RESPONSE	MEANS	ACTION
429, 500, 529, connection	Transient: busy or limited	Back off and retry; SDK does it twice
400, 401, 403, 404, 413	Your request or key is wrong	Fix it; never retry unchanged
429 at the spend cap, 402	Money, not load	Stop and escalate; no retry-after
504 on a long request	Timed out while processing	Stream it, or use a batch

CURRENCY CHECK

Catch typed exception classes, most specific first, and log the request ID from every error.

HTTP 200 is not the same as success

- **max_tokens:** truncated. Raise the limit or continue; an unfinished tool_use must be retried.
- **refusal:** a normal 200, not an error. Read stop_details, then fall back.
- **model_context_window_exceeded:** treat the response as truncated.
- **Empty end_turn:** usually text added right after tool results. Send tool_result blocks only.
- **Streams:** an error event can arrive after the 200 has been sent.

> THE TIEBREAKER
>
> Check stop_reason before you parse. Many "the model is broken" reports are unhandled stop reasons.

A Claude call you can debug later

```python
def triage(ticket, cfg):
    req = build_request(ticket, cfg)              # prompt and model from config
    try:
        resp = client.messages.create(**req)      # SDK retries 429 and 5xx twice
    except anthropic.BadRequestError as e:        # our bug: never retry a 400
        log_error(ticket.id, e.request_id, "bad_request")
        raise
    log_trace(ticket.id, resp._request_id, cfg.prompt_version, resp.model,
              resp.usage, resp.stop_reason, raw=resp.content)
    if resp.stop_reason != "end_turn":            # 200 is not success
        return route_to_person(ticket, reason=resp.stop_reason)
    return validate(parse_triage(resp))           # integration layer from here
```

5-7 Typed error class; request ID logged; a 400 is not retried.

8-9 The trace: request ID, prompt version, model, usage, raw output.

10-11 Branch on the stop reason before parsing anything.

THE TAKEAWAY

Log the raw response before parsing. It is the line between the model and your code.

What every Claude call should log

- **Request ID,** from the response or the error. Support asks for it first.
- **Model ID, prompt version and config flags,** so you know what actually ran.
- **Usage and latency:** input, output and cache tokens; time to first token.
- **Stop reason, tool calls and tool results,** in order.
- **Raw output before parsing,** plus the validation result. Personal data masked.

> THE TIEBREAKER
>
> If you cannot replay a failing request exactly, you are guessing.

Traces without building them

	SDK TRACING	API TRACING
Set up	Register a tracer provider in code	Add a destination in the Console
Covers	Clients that use the SDK	Any client; admin sets it up
Spans	anthropic.messages.create, tool spans	Model requests, server tools, sessions
Content	metadata_only by default	Metadata only by default

CURRENCY CHECK

OpenTelemetry export, public beta since 14 Oct 2026. Best effort: spans can be lost or duplicated.

Integration layer or model?

01
Reproduce
Find the failing request ID and its trace.

02
Raw output
Read Claude's response before any parsing.

03
Raw is right
But the app is wrong: parsing, mapping or display.

04
Raw is wrong
Check the request: was the right context sent?

05
Request right
Model layer: prompt, examples, model. Prove with evals.

THE TRAP

Context assembly is your code. Missing context looks like a model fault, but it is integration.

Symptom, then where to look

SYMPTOM	CHECK FIRST	USUALLY
Raw output right, screen wrong	Parser and field mapping	Integration
Answer ignores a document	Was it in the request?	Integration: retrieval
Same input, different labels	Category definitions, examples	Prompt
Quality drops on one date	Deploys, model, prompt, data	Whatever changed
Same tool called again and again	The tool's error result	Tool contract

ON THE EXAM

Swapping the model is almost never the first move. It hides the evidence.

Cluster before you fix

- **Collect failures** from traces, user reports and reviewer corrections.
- **Group them** by what they share: input type, length, turn count, model, prompt version.
- **Line up the timeline:** a deploy, model change or data change at the start?
- **Change one thing,** then re-run the cluster and the full eval set.
- **Keep the cases:** every confirmed failure becomes an eval case.

> RULE 2
>
> Evidence before change: diagnose from the failing requests, then change one thing.

Copperlet: "just lower the temperature"

BEFORE

A dial, and no evidence

- Agents re-labelled the trade on about 4% of tickets
- Proposal: lower the temperature (rejected in m02)
- Enums fixed the vocabulary, not the choice (m07)
- Nobody had read a single trace

AFTER

Traces, then prompt v8

- 300 re-labelled tickets traced: model, prompt, stop reasons normal
- 7 in 10 were leaks from a boiler, radiator or cylinder
- v7: heating = boilers, plumbing = leaks. Both fit
- v8: boundary rule, 3 examples; agreement 0.78 to 0.97

SETTLED

Quirk 12 settled. v8 ships through the 5% canary flag from module 16.

Copperlet's traces, looking back

PROBLEM	WHAT THE TRACE SHOWED	LAYER
Quirk 4, forgetful assistant	Tenant name only in turn one; input growing	Context
Quirk 13, Friday upgrade	Format changed the minute the model ID did	The change
Quirk 10, the loop	30 identical lookups, each an empty string	Tool contract
Quirk 8, regex for JSON	Crashes matched stop reason max_tokens	Integration

THE PATTERN

Four problems, four different layers. None was fixed by swapping the model.

Recovery strategies, by design

STRATEGY	WHEN	EXAMPLE
Retry	Transient: 429, 5xx, connection	Backoff honouring retry-after
Repair	Output fails validation	Send the error back, once
Fall back	Refusal, outage, bad new version	Another model; last good prompt
Degrade	Part of the job fails	Partial answer, clearly flagged
Fail cleanly	Nothing else works	Clear message; queue for a person

RULE 7

Failure is a designed path. Decide the order before production, not during the incident.

> A support assistant has given wrong refund-window answers in twelve complaints this week.
>
> A teammate wants to rewrite the system prompt today.

What should the team do first?

A Rewrite the system prompt with the refund rules in capitals

B Move the assistant to the most capable model available

C Set the temperature to zero on every request

D Pull the traces of the reported chats and compare them

◆ PAUSE AND DECIDE

What should the team do first?

A Rewrite the system prompt with the refund rules in capitals

A change before any diagnosis.

B Move the assistant to the most capable model available

Hides the evidence; may not help.

C Set the temperature to zero on every request

A sampling dial; current models reject it.

D Pull the traces of the reported chats and compare them

Evidence before change.

THE SIGNAL

"First" questions reward evidence. Find the cause, then fix the layer it points to.

> An invoice extractor shows the wrong total for some invoices in the finance dashboard.
>
> For those invoices, the logged raw responses from Claude hold the correct totals.

Where is the fault, and what is next?

A Integration: inspect the parsing and field mapping

B The model: add worked examples of totals to the prompt

C The model: move to a larger model for invoices

D The input: re-scan the invoices at a higher resolution

◆ PAUSE AND DECIDE

Where is the fault, and what is next?

A Integration: inspect the parsing and field mapping
Raw right, screen wrong.

B The model: add worked examples of totals to the prompt
The model already got it right.

C The model: move to a larger model for invoices
Fixes a layer that is not broken.

D The input: re-scan the invoices at a higher resolution
The input was read correctly.

THE SIGNAL

Raw output right, application wrong: the fault is between the response and the screen.

> An insurance tool summarises claim descriptions, some describing injuries in detail. A small share of responses arrive with HTTP 200 but no usable text.
>
> The tool stores them as blank summaries.

What should the developer add?

A Retry each blank with the same model until text arrives

B Branch on the stop reason; on refusal, fall back

C A system prompt line telling Claude never to refuse

D An alert when blank summaries pass a set level

◆ PAUSE AND DECIDE

What should the developer add?

A Retry each blank with the same model until text arrives

Same request, same refusal, more cost.

B Branch on the stop reason; on refusal, fall back

Handles the 200 that is not success.

C A system prompt line telling Claude never to refuse

The prompt cannot switch off refusals.

D An alert when blank summaries pass a set level

Notices the damage; does not prevent it.

THE SIGNAL

A refusal is a stop reason on a 200, not an HTTP error. Check it before storing anything.

> A nightly job logs some 529 overloaded errors, and a steady stream of 400 errors from one service since yesterday's deploy.
>
> The team plans to raise max_retries to ten for everything.

What is the best response?

A Raise the retry count to ten, as planned

B Move every request in the job to a batch

C Open one support ticket covering both kinds of error

D Keep backoff for overloads; fix the deploy's bug

◆ PAUSE AND DECIDE

What is the best response?

A Raise the retry count to ten, as planned

Retries a broken request ten times.

B Move every request in the job to a batch

Does not fix the broken request.

C Open one support ticket covering both kinds of error

The 400s are your own bug.

D Keep backoff for overloads; fix the deploy's bug

Transient retried; client error fixed.

THE SIGNAL

Two families, two fixes. A 400 is your request, and retrying it unchanged only wastes time.

> A team is launching a Claude feature. It wants to debug production failures later, without storing more personal data than it needs.
>
> It is deciding what every call should log.

Which two should every call log? Choose two.

A The request ID, model ID and prompt version

B The full prompt and reply of every call, kept indefinitely

C The stop reason, token usage and latency

D Only the calls that raised an HTTP error

E Nothing, since Anthropic keeps a copy of every request

◆ PAUSE AND DECIDE

Which two should every call log? Choose two.

A The request ID, model ID and prompt version
Identifies and replays what ran.

B The full prompt and reply of every call, kept indefinitely
Far more personal data than needed.

C The stop reason, token usage and latency
Shows truncation, refusals and cost.

D Only the calls that raised an HTTP error
Misses every failure on a 200.

E Nothing, since Anthropic keeps a copy of every request
Not your log, and may not be kept.

THE SIGNAL

Log enough to replay a failure: identifiers and versions, plus how the call ended.

> A research agent built on the Agent SDK sometimes runs for 40 turns and exhausts its budget. Nobody knows which step goes wrong.
>
> Its logs hold only the final answer.

What should the team do first?

A Cap the agent at ten turns, and close the issue as fixed

B Tell the agent in its prompt to be more efficient

C Export traces and read the failing runs' tool spans

D Rebuild the agent in a different framework

◆ PAUSE AND DECIDE

What should the team do first?

A Cap the agent at ten turns, and close the issue as fixed

Hides the symptom; cause unknown.

B Tell the agent in its prompt to be more efficient

A guess, not a diagnosis.

C Export traces and read the failing runs' tool spans

Shows which step loops or stalls.

D Rebuild the agent in a different framework

Rebuilding before diagnosing.

THE SIGNAL

Agents fail between the first prompt and the final answer. Traces show the steps in between.

> A ticket router's accuracy dropped on 3 March. Traces show the model and the prompt version unchanged.
>
> That morning, the knowledge-base retrieval service was redeployed, and now returns 2 chunks instead of 8.

Where should the team fix it?

A Revert the retrieval change; test what reaches it

B Add more labelled examples to the router's prompt

C Upgrade the router to the newest model available

D Raise max_tokens so the router can explain itself

◆ PAUSE AND DECIDE

Where should the team fix it?

A Revert the retrieval change; test what reaches it

Fixes what changed, and guards it.

B Add more labelled examples to the router's prompt

Prompt was not the change.

C Upgrade the router to the newest model available

Model was not the change.

D Raise max_tokens so the router can explain itself

Output length is not the problem.

THE SIGNAL

Line up the timeline. The fix belongs to whatever changed when quality dropped.

> A shop translates product pages with Claude. During a regional outage, the primary model returns 529 errors for an hour.
>
> The shop shows error pages to customers.

What recovery design fits best?

A Retry immediately in a tight loop until it succeeds

B Keep showing the error page, since outages are rare

C Remove AI translation until uptime is guaranteed

D Back off and retry, then fall back to a cached copy

◆ PAUSE AND DECIDE

What recovery design fits best?

A Retry immediately in a tight loop until it succeeds

Adds load; ignores backoff.

B Keep showing the error page, since outages are rare

Accepts a failure it could design out.

C Remove AI translation until uptime is guaranteed

Permanent avoidance of a fixable risk.

D Back off and retry, then fall back to a cached copy

Retry, then fall back, by design.

THE SIGNAL

Recovery has an order: retry the transient, then fall back, then degrade or fail cleanly.

Module 30 in five lines

- **Name the family:** HTTP error, stop reason, integration fault or model output.
- **200 is not success:** branch on stop_reason before parsing.
- **Trace every call:** request ID, model, prompt version, usage, raw output.
- **Isolate the layer:** raw output first, then the request, then the prompt.
- **Recover by design:** retry, repair, fall back, degrade, fail cleanly.

COMING UP

Next, Part 10: review – the confusion pairs the exam loves to test.